

00 / MATERIAL DE ESTUDO

Breviário de fundamentos

IA para iniciantes: agentes, RAG, Transformers, LLMs, redes neurais e algoritmos de ML.

TURMA FUNDADORA | AGO-SET 2026
INVERTED AGENTIC ORCHESTRATION



Este é o volume introdutório dos mesmos seis temas do breviário técnico: Agentes, RAG, Transformers, LLMs, Redes Neurais e Algoritmos de ML. A sequência começa pelo vocabulário básico e termina mostrando como tudo se conecta em uma aplicação simples de seguros.

Leia um dia por vez. Não tente decorar tudo; ao final de cada item, procure explicar a ideia com suas próprias palavras.

Visão geral

MAPA CONCEITUAL

O diagrama original acompanha a sequência: flowchart LR D[Dados] --> M[Modelo de ML ou rede neural] M --> T[Transformer] T --> L[LLM] F[Documentos] --> R[RAG] L --> A[Agente] R --> A A --> U[Usuário ou sistema]

Dia 1 — o vocabulário essencial

Agentes

Um agente é um programa que recebe um objetivo, observa informações disponíveis, escolhe uma ação e usa o resultado para decidir o próximo passo; em IA generativa, um LLM pode fazer parte do agente como componente de decisão, mas o agente também precisa de ferramentas, regras, memória e um ambiente onde suas ações tenham efeito. Referência introdutória: AgentBench — Evaluating LLMs as Agents — <https://arxiv.org/abs/2308.03688>.

RAG

RAG significa Retrieval-Augmented Generation, ou geração aumentada por recuperação: antes de responder, o sistema busca trechos relevantes em documentos e entrega esses trechos ao LLM como contexto; isso é útil quando a informação muda com frequência, como regras de produto, apólices ou procedimentos internos, sem precisar treinar novamente o modelo a cada alteração. Referência: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks — <https://arxiv.org/abs/2005.11401>.

Arquitetura Transformer

Transformer é uma arquitetura de rede neural criada para trabalhar com sequências, como frases, usando atenção para descobrir quais partes da entrada são mais importantes umas para as outras; ela substituiu grande parte do uso de redes recorrentes em linguagem porque permite processar muitos tokens em paralelo durante o treinamento. Referência: Attention Is All You Need — <https://arxiv.org/abs/1706.03762>.

LLMs

LLM significa Large Language Model, ou modelo de linguagem grande; ele aprende padrões de texto observando muitos exemplos e, durante a geração, calcula qual próximo token é mais provável dado o texto anterior, repetindo esse processo até formar uma resposta. Referência introdutória: Language Models are Few-Shot Learners — <https://arxiv.org/abs/2005.14165>.

Redes neurais

Uma rede neural é um conjunto de camadas que transforma uma entrada em uma saída por meio de números chamados pesos; durante o treinamento, os pesos são ajustados para reduzir os erros do modelo, permitindo que ele aprenda relações complexas em texto, imagem, áudio ou dados tabulares. Referência: Deep Learning — <https://www.deeplearningbook.org/>.

Algoritmos de ML

Um algoritmo de Machine Learning aprende uma relação entre dados de entrada e um resultado, em vez de receber todas as regras escritas manualmente; regressão logística, árvores de decisão, Random Forest e gradient boosting são exemplos clássicos, especialmente úteis quando os dados já estão organizados em colunas como idade, valor, produto ou histórico. Referência: Google Machine Learning Crash Course — <https://developers.google.com/machine-learning/crash-course>.

Dia 2 — dados, tokens e representações

Agentes

Um agente precisa de um loop básico: objetivo □ observação □ decisão □ ação □ novo resultado; por exemplo, ao analisar um sinistro, ele pode receber a pergunta, consultar os dados do caso, buscar uma regra, propor uma classificação e pedir aprovação humana, em vez de produzir apenas um texto sem conexão com sistemas externos. Referência: ReAct — Synergizing Reasoning and Acting in Language Models — <https://arxiv.org/abs/2210.03629>.

RAG

Antes de buscar documentos, o texto costuma ser dividido em pedaços chamados chunks e transformado em uma representação numérica chamada embedding; documentos e pergunta com representações próximas são candidatos à recuperação, mas proximidade não significa verdade, por isso o sistema ainda deve filtrar documentos corretos e verificar a resposta. Referência introdutória: Dense Passage Retrieval for Open-Domain Question Answering — <https://arxiv.org/abs/2004.04906>.

Arquitetura Transformer

Um Transformer não recebe palavras diretamente: o texto é dividido em tokens e cada token vira um vetor; a atenção compara esses vetores para estimar relações de contexto, enquanto outras camadas transformam a representação, fazendo com que o significado de um token dependa também dos tokens ao redor. Referência visual: The Illustrated Transformer — <https://jalammar.github.io/illustrated-transformer/>.

LLMs

Token é uma unidade de texto usada pelo modelo e pode ser uma palavra inteira, parte de uma palavra, pontuação ou símbolo; o número de tokens influencia o limite de contexto, o custo e a velocidade, e por isso “uma página de texto” não corresponde necessariamente ao mesmo tamanho para todos os modelos. Referência: SentencePiece — A simple and language independent subword tokenizer — <https://arxiv.org/abs/1808.06226>.

Redes neurais

Embeddings são vetores que representam itens em um espaço numérico: textos parecidos tendem a ficar próximos, assim como imagens ou usuários com características semelhantes; eles são úteis para busca semântica, recomendação e classificação, mas só têm significado porque foram aprendidos a partir de dados e de um objetivo específico. Referência: Deep Learning — <https://www.deeplearningbook.org/>, capítulos sobre representações.

Algoritmos de ML

Em aprendizado supervisionado, cada exemplo geralmente tem entradas (x) e uma resposta esperada (y), como dados do segurado e ocorrência de fraude; o algoritmo usa exemplos conhecidos para aprender e depois recebe exemplos novos, enquanto aprendizado não supervisionado procura padrões sem uma resposta pronta, como grupos ou anomalias. Referência: Google Machine Learning Crash Course — <https://developers.google.com/machine-learning/crash-course>.

Dia 3 — como um modelo aprende

Agentes

O LLM dentro de um agente não “sabe” automaticamente quais ações são permitidas: as ferramentas disponíveis, seus parâmetros e suas regras precisam ser descritos pelo sistema; uma boa ferramenta tem uma finalidade clara, entrada validada e resultado estruturado, como `buscar_apolice(numero)` em vez de uma ferramenta genérica que pode fazer qualquer coisa. Referência: Toolformer — Language Models Can Teach Themselves to Use Tools — <https://arxiv.org/abs/2302.04761>.

RAG

O fluxo básico de RAG tem duas fases: na indexação, documentos são limpos, divididos e armazenados com seus embeddings; na consulta, a pergunta também é transformada, documentos candidatos são recuperados e os melhores trechos são colocados no prompt do LLM, de modo que uma mudança no documento pode aparecer no sistema sem alterar os pesos do modelo. Referência: Retrieval-Augmented Generation — <https://arxiv.org/abs/2005.11401>.

Arquitetura Transformer

Durante o treinamento de um Transformer autoregressivo, o modelo recebe uma sequência e tenta prever o token seguinte; uma máscara causal impede que ele veja o futuro, mas o treinamento continua eficiente porque muitos tokens podem ser processados em paralelo, enquanto a geração é sequencial token a token. Referência: Attention Is All You Need — <https://arxiv.org/abs/1706.03762>.

LLMs

Treinamento e uso são coisas diferentes: no treinamento, o modelo altera seus pesos para diminuir a perda em muitos exemplos; na inferência, os pesos ficam congelados e o modelo apenas calcula uma resposta para uma nova entrada, embora essa entrada possa incluir instruções, exemplos e documentos recuperados. Referência: Language Models are Few-Shot Learners — <https://arxiv.org/abs/2005.14165>.

Redes neurais

O treinamento costuma repetir quatro passos: fazer uma previsão, calcular a diferença entre previsão e resposta, calcular como cada peso contribuiu para o erro e atualizar os pesos; esse processo é chamado, respectivamente, de forward pass, loss, backpropagation e otimização, e é a base de praticamente todo deep learning moderno. Referência: Deep Learning — <https://www.deeplearningbook.org/>, capítulos sobre backpropagation.

Algoritmos de ML

Os dados normalmente são divididos em treino, validação e teste: treino ajusta o modelo, validação ajuda a escolher configurações e teste mede o desempenho final em exemplos não usados nas decisões; se o desempenho no treino é muito melhor que no teste, pode existir overfitting, quando o modelo memoriza detalhes dos exemplos em vez de aprender um padrão generalizável. Referência: Understanding deep learning requires rethinking generalization — <https://arxiv.org/abs/1611.03530>.

Dia 4 — os componentes trabalhando juntos

Agentes

A diferença entre chatbot e agente aparece quando existe ação: um chatbot pode explicar como consultar uma apólice, enquanto um agente pode consultar a apólice, verificar documentos, registrar uma tarefa e encaminhar o caso; quanto mais consequências uma ação tiver, mais importante se torna separar sugestão, execução autorizada e aprovação humana. Referência: ReAct — <https://arxiv.org/abs/2210.03629>.

RAG

RAG não elimina alucinação: ele apenas fornece evidências adicionais ao gerador; se o retriever não encontrar o documento certo, se o chunk estiver incompleto ou se o LLM ignorar o contexto, a resposta ainda pode estar errada, motivo pelo qual sistemas sérios precisam permitir “não encontrei evidência suficiente” como resposta válida. Referência: REALM — Retrieval-Augmented Language Model Pre-Training — <https://arxiv.org/abs/2002.08909>.

Arquitetura Transformer

A atenção pode ser entendida como uma pergunta feita por cada token aos demais: (Q) representa o que ele procura, (K) representa o que cada token oferece para comparação e (V) contém a informação que será combinada; várias cabeças fazem esse processo em subespaços diferentes, capturando relações distintas na mesma sequência. Referência visual e técnica: The Illustrated Transformer — <https://jalammar.github.io/illustrated-transformer/> e Attention Is All You Need — <https://arxiv.org/abs/1706.03762>.

LLMs

Um LLM pode usar conhecimento aprendido nos pesos, exemplos fornecidos no prompt ou informações recuperadas externamente; essas fontes não são equivalentes: pesos podem estar desatualizados, exemplos consomem contexto e RAG depende da qualidade da busca, então escolher entre prompt, RAG e fine-tuning é uma decisão de engenharia, não apenas de preferência de modelo. Referência: RAG — <https://arxiv.org/abs/2005.11401>.

Redes neurais

Função de ativação, como ReLU, introduz não linearidade para que várias camadas consigam representar relações que uma transformação linear única não consegue; sem não linearidades, empilhar muitas camadas ainda equivaleria a uma única transformação linear, limitando a capacidade da rede. Referência: Deep Learning — <https://www.deeplearningbook.org/>, capítulos sobre redes feed-forward.

Algoritmos de ML

Árvores de decisão fazem perguntas sucessivas sobre features, como “valor maior que X?” ou “produto é Y?”, até chegar a uma previsão; elas são fáceis de explicar e lidam bem com relações não lineares, mas uma árvore isolada pode memorizar os dados, motivo pelo qual ensembles como Random Forest e XGBoost são muito usados. Referências: Random Forests — <https://link.springer.com/article/10.1023/A:1010933404324> e XGBoost — <https://arxiv.org/abs/1603.02754>.

Dia 5 — qualidade, erros e confiança

Agentes

Um agente precisa ser avaliado pelo resultado da tarefa e pelo caminho usado para chegar lá: acertar a resposta final não basta se ele chamou uma ferramenta proibida, gastou recursos sem limite ou alterou dados incorretamente; por isso avaliações de agentes observam sucesso, uso de ferramentas, número de passos, recuperação de erros e segurança das ações. Referência: AgentBench — Evaluating LLMs as Agents — <https://arxiv.org/abs/2308.03688>.

RAG

A qualidade de RAG pode ser dividida em pelo menos três perguntas: o trecho recuperado é relevante para a pergunta, a resposta está apoiada no trecho e a resposta realmente atende ao usuário? Separar essas perguntas ajuda a descobrir se o problema está na busca ou na geração, em vez de tentar corrigir tudo mudando o prompt. Referência: ARES — An Automated Evaluation Framework for Retrieval-Augmented Generation Systems — <https://aclanthology.org/2024.naacl-long.20/>.

Arquitetura Transformer

Quanto maior a sequência, mais relações de atenção podem precisar ser calculadas e mais memória é usada; essa é uma razão para existirem técnicas de cache, atenção eficiente, janelas de contexto e modelos especializados, e também explica por que “aceita um contexto grande” não significa que o modelo use todos os trechos igualmente bem. Referência: FlashAttention — <https://arxiv.org/abs/2205.14135>.

LLMs

Temperatura controla a aleatoriedade da geração: valores mais baixos tendem a escolher respostas mais previsíveis, enquanto valores mais altos exploram tokens menos prováveis; isso muda estilo e diversidade, mas não transforma um modelo incorreto em um modelo factual, porque o conhecimento e a evidência continuam sendo os fatores principais. Referência: Language Models are Few-Shot Learners — <https://arxiv.org/abs/2005.14165>.

Redes neurais

Confiança não é a mesma coisa que acurácia: uma rede pode acertar muitas vezes e ainda atribuir probabilidade exagerada às próprias previsões; calibração procura fazer uma previsão de 80% corresponder aproximadamente a um evento que ocorre em 80% dos casos semelhantes, algo importante para decidir quando automatizar ou pedir revisão. Referência: On Calibration of Modern Neural Networks — <https://arxiv.org/abs/1706.04599>.

Algoritmos de ML

As métricas devem combinar com o problema: accuracy pode esconder uma classe rara, precision mede quanto dos alertas são corretos, recall mede quanto dos casos relevantes foram encontrados e F1 combina precision e recall; em fraude ou sinistros, também é preciso considerar o custo de falso positivo e falso negativo, não apenas uma métrica estatística. Referência: Google Machine Learning Crash Course — <https://developers.google.com/machine-learning/crash-course>.

Dia 6 — noções de produção

Agentes

Em produção, um agente precisa de limites como número máximo de passos, tempo total, tokens e custo, além de fallback e circuit breaker; isso evita que uma falha de interpretação vire uma sequência infinita de chamadas ou uma ação repetida, e transforma um protótipo inteligente em um componente operável. Referência: AgentBench — <https://arxiv.org/abs/2308.03688>.

RAG

Os documentos de um RAG precisam de controle de acesso e de versão: um usuário só pode receber informações que está autorizado a ver, e uma regra antiga não deve ser recuperada como se ainda estivesse vigente; metadados como produto, data, região e perfil de acesso são tão importantes quanto o embedding. Referência: REALM — <https://arxiv.org/abs/2002.08909>.

Arquitetura Transformer

Na geração token a token, o KV cache guarda parte dos cálculos anteriores para evitar refazer todo o histórico em cada passo; ele acelera a geração, mas consome memória e cresce com o tamanho do contexto e a quantidade de requisições simultâneas, tornando serving e gestão de memória parte do desenho da arquitetura. Referência: PagedAttention — <https://arxiv.org/abs/2309.06180>.

LLMs

O custo de um LLM não é apenas o preço do modelo: também depende de tokens de entrada, tokens de saída, tamanho do contexto, concorrência, latência desejada, retries e chamadas de ferramentas; por isso, um modelo menor com bom roteamento ou RAG pode ser melhor para uma aplicação do que um modelo maior usado em todas as requisições. Referência: Training Compute-Optimal Large Language Models — <https://arxiv.org/abs/2203.15556>.

Redes neurais

Monitoramento de modelo verifica se os dados e resultados mudaram depois do treinamento: drift de dados ocorre quando a distribuição das entradas muda, enquanto degradação de performance ocorre quando a relação entre entrada e resultado muda; acompanhar ambos ajuda a saber quando recalibrar, retreinar ou interromper uma automação. Referência: Deep Learning — <https://www.deeplearningbook.org/>, capítulos sobre generalização e regularização.

Algoritmos de ML

Modelos clássicos continuam importantes porque dados tabulares, poucos exemplos rotulados, necessidade de explicação e baixa latência muitas vezes favorecem árvores ou regressão em vez de uma rede neural grande; a escolha correta começa pela natureza do dado e pelo custo do erro, não pela novidade do algoritmo. Referência: XGBoost — <https://arxiv.org/abs/1603.02754>.

Dia 7 — síntese com um exemplo de seguros

Agentes

Para um agente de análise inicial de sinistro, uma arquitetura básica pode receber o relato, consultar a apólice, buscar procedimentos, classificar o caso e sugerir o próximo passo, mas deve enviar decisões de alto risco para um humano; o LLM ajuda a interpretar e planejar, enquanto regras e sistemas oficiais controlam permissões e fatos. Referências: ReAct — <https://arxiv.org/abs/2210.03629> e AgentBench — <https://arxiv.org/abs/2308.03688>.

RAG

Nesse caso, RAG pode recuperar a cobertura aplicável, a versão vigente das condições gerais e o procedimento de regulação, entregando essas fontes ao LLM para produzir uma explicação citada; se não houver evidência suficiente ou os documentos forem conflitantes, o comportamento seguro é pedir revisão ou mais informação, não inventar uma conclusão. Referência: Retrieval-Augmented Generation — <https://arxiv.org/abs/2005.11401>.

Arquitetura Transformer

O Transformer é o mecanismo interno que ajuda o LLM a relacionar partes do relato, dos documentos e das instruções dentro do contexto, mas ele não conhece por si só a verdade da apólice nem executa a consulta; essa separação ajuda a entender a arquitetura: o modelo processa linguagem, enquanto RAG e tools fornecem informação e ação. Referência: Attention Is All You Need — <https://arxiv.org/abs/1706.03762>.

LLMs

O LLM pode resumir o relato, extrair campos, explicar uma regra e propor uma classificação, mas sua resposta é probabilística e pode conter erro; por isso, em uma aplicação real, a resposta deve ser limitada por contexto autorizado, formato estruturado, validações, métricas e possibilidade de abstenção. Referência: Training Language Models to Follow Instructions with Human Feedback — <https://arxiv.org/abs/2203.02155>.

Redes neurais

Uma rede neural pode ajudar em tarefas específicas, como classificar imagens de danos ou extrair informações de documentos, porque aprende padrões em exemplos; entretanto, “usar uma rede neural” não define a solução inteira, pois ainda é necessário decidir dados de treinamento, rótulos, métrica, limiar de decisão, tratamento de incerteza e processo de revisão. Referência: Deep Learning — <https://www.deeplearningbook.org/>.

Algoritmos de ML

Um sistema de seguros pode combinar algoritmos: regressão ou XGBoost para risco tabular, um modelo de ranking para ordenar documentos relevantes e detecção de anomalias para priorizar investigação; cada modelo resolve uma parte diferente do problema, e o resultado final precisa ser avaliado pelo impacto no negócio e pelo risco operacional, não apenas pela precisão isolada. Referências: XGBoost — <https://arxiv.org/abs/1603.02754> e Isolation Forest — <https://seppe.net/aa/papers/iforest.pdf>.

Ao terminar a semana

Você deve conseguir explicar, em linguagem simples:

- o que é um agente e por que ele precisa de ferramentas e limites;
- o que RAG acrescenta a um LLM e quais problemas ele não resolve;
- como um Transformer usa atenção;
- como um LLM aprende e gera tokens;
- como uma rede neural é treinada;
- quando um algoritmo clássico de ML pode ser melhor que uma rede neural.

Depois disso, use o breviário técnico avançado para aprofundar os mesmos conceitos.